Taller de Visión Artificial
Control 2

Maikol Julián Pulido | Ingeniería Electrónica | Escuela Colombiana de Ingeniería Julio Garavito

Continuación práctica y evidencia reproducible | 14 de octubre de 2026

Alcance y estado de entrega

Se implementan los puntos 1, 2, 6, 7, 9 y 10 con Python/OpenCV y NumPy. Se conserva íntegra la investigación aportada: su PDF original aparece al final como anexo, sin reescritura. Las figuras geométricas del punto 9 son entradas sintéticas solicitadas por la consigna; los cuadros de los puntos 2 y 10 proceden del video real existente.

El clip Otsu-inverso original contiene 458 cuadros decodificados, 320 × 240 píxeles y 30.537 cuadros/s; duración nominal 14.998 s. Se extraen 40 cuadros distintos y se calcula la DFT completa de cada uno.

Punto	Evidencia / estado
1	Dos métodos por fila y comparación numérica
2	40 PNG, índices y tiempos; 20 muestras orientadas a curvas
6	Máscaras 3x3 y distancias en px; cm pendientes de calibración
7	Error lateral y PWM propuesto; sin prueba en motores
9	Círculo, cuadrado, línea: magnitud, fase y ejemplo unwrap
10	40 DFT; cociente, diferencia de fase, inversa y desplazamiento

No hay una referencia métrica de dimensiones conocidas ni calibración de cámara en los materiales utilizados. Por tanto, no se presentan centímetros, velocidad real ni distancia recorrida. El PWM es una propuesta offline y no demuestra que el robot siga la pista.

Trabajo previo, entradas y trazabilidad

Se inspeccionó Taller_Procesamiento_Imagenes: punto1.py recorta ideal.mp4 desde t=0 durante 15 s y genera gris/RGB; punto1_otsu.py genera las versiones Otsu original e inversa y sus variantes con gamma 2.2. También existen scripts de diferencias, filtrado y máscaras. Estos scripts se preservan en previo/. No se confunde su numeración con los puntos del nuevo taller.

Se revisó adicionalmente Robot_OpenCV/deteccion_camino.py: implementa segmentos por filas para un stream de ESP32-CAM. No se ejecutó esa conexión ni se modificó ese proyecto. La continuación se aloja exclusivamente en Taller_Procesamiento_Imagenes/taller3.

Se compararon visualmente los cuadros 0, 90, 180, 270, 360 y 450 del RGB, Otsu-inverso original y Otsu-inverso con gamma. Se eligió la versión original porque la variante gamma incorpora más fondo blanco en esas muestras. La elección es visual y no una evaluación de exactitud con anotaciones de referencia.

Entrada de cálculo: entradas/video_otsu_inverso.mp4, copia de resultados/taller2/clip_original_otsu_inv.mp4. Se vuelve a binarizar a 127 únicamente para quitar niveles grises introducidos por la compresión del MP4; no se aplica un segundo Otsu inverso, que invertiría nuevamente la polaridad. Blanco representa la línea oscura de la escena.

SHA-256 de la entrada: f366faa84168e9d4cb47212fc3f12c618643b8dae59d119fcbc24497ab553ce0

Entorno ejecutado: Python 3.12.14; OpenCV 5.0.0; NumPy 2.3.5; Matplotlib 3.11.2. Índices base cero, x crece a la derecha e y hacia abajo. Los tiempos i/FPS se refieren al clip y son nominales.

La investigación original usa “transformada de distancia” al discutir distancia entre dos puntos. Precisión complementaria: distanceTransform entrega la distancia de cada píxel no nulo al cero más cercano; la distancia entre dos coordenadas especificadas se obtiene directamente con sus diferencias. Esta aclaración no modifica el anexo.

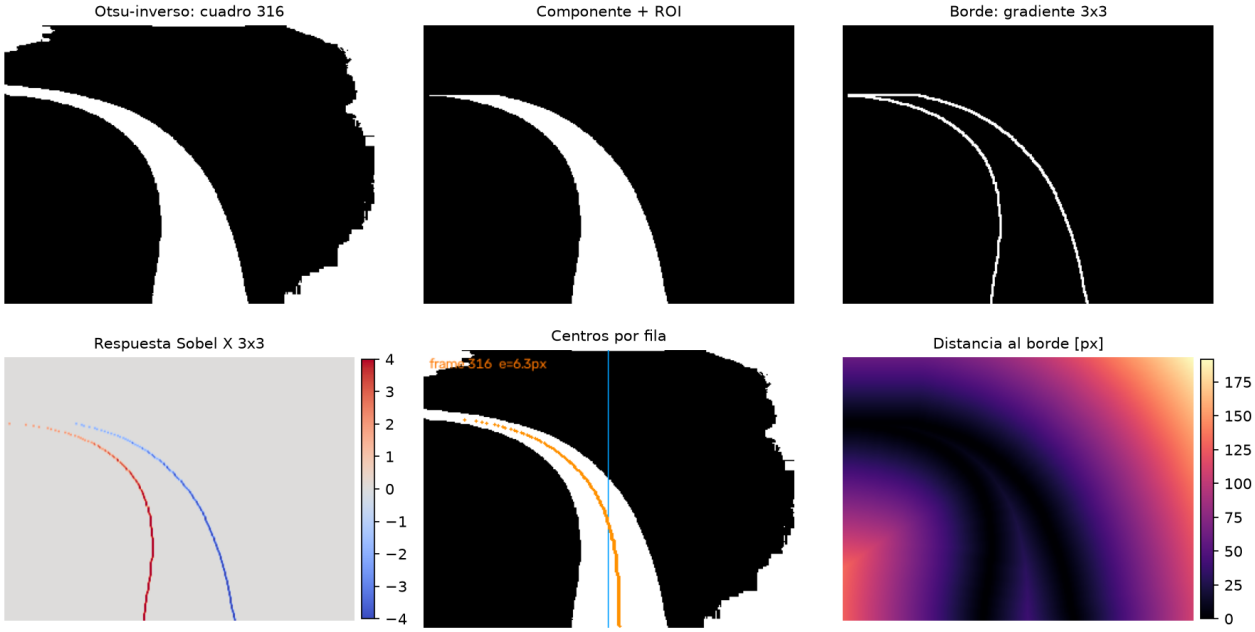
1. Centro del carril por fila: dos opciones

Opción A, transiciones de borde: se filtra la máscara con Sobel X de 3x3. En cada fila se localiza un tramo blanco continuo delimitado por una transición de entrada y una de salida. Con extremos inclusivos xL y xR, el centro es c(y)=(xL+xR)/2 y el ancho es xR-xL+1. Si hay varios tramos, se escoge el centro más próximo al de la fila inferior; al iniciar se usa el centro de imagen.

Opción B, momento de primer orden: sobre la componente seleccionada, cM(y)=sum(x B(x,y))/sum(B(x,y)), con B binaria. Es el centroide de los píxeles blancos de la fila. Si no hay píxeles, el centro es indefinido y no se sustituye por cero. En un único tramo uniforme ambos métodos coinciden; agujeros y múltiples tramos pueden separarlos.

La implementación aplica apertura y cierre con unos 3x3 y bordes replicados. Selecciona la mayor componente conectada que alcanza el 20% inferior, dentro de y>=0.25H. Rechaza tramos que tocan los laterales, anchos fuera de [3,0.65W] y saltos de centro superiores a 0.12W. Esta hipótesis es adecuada para la cinta única observada; cruces, bifurcaciones o una sombra conectada pueden invalidarla.

Resultado: diferencia absoluta media entre ambos centros, promediada primero por cuadro y luego sobre los 40 cuadros: 3.1914 px. La concordancia entre métodos no equivale a exactitud respecto al centro físico; no existe anotación manual de referencia.

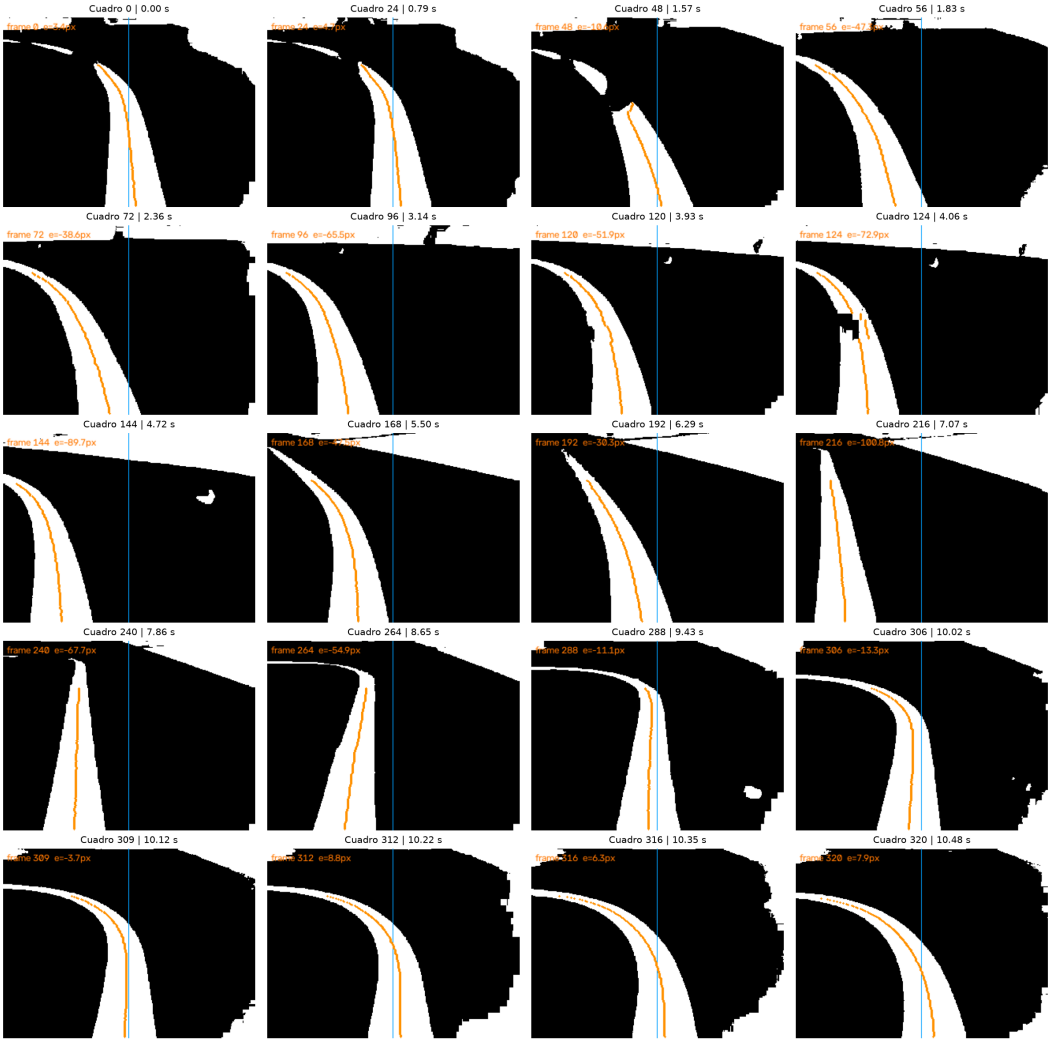


2. Extracción de 40 cuadros, con énfasis en curvas

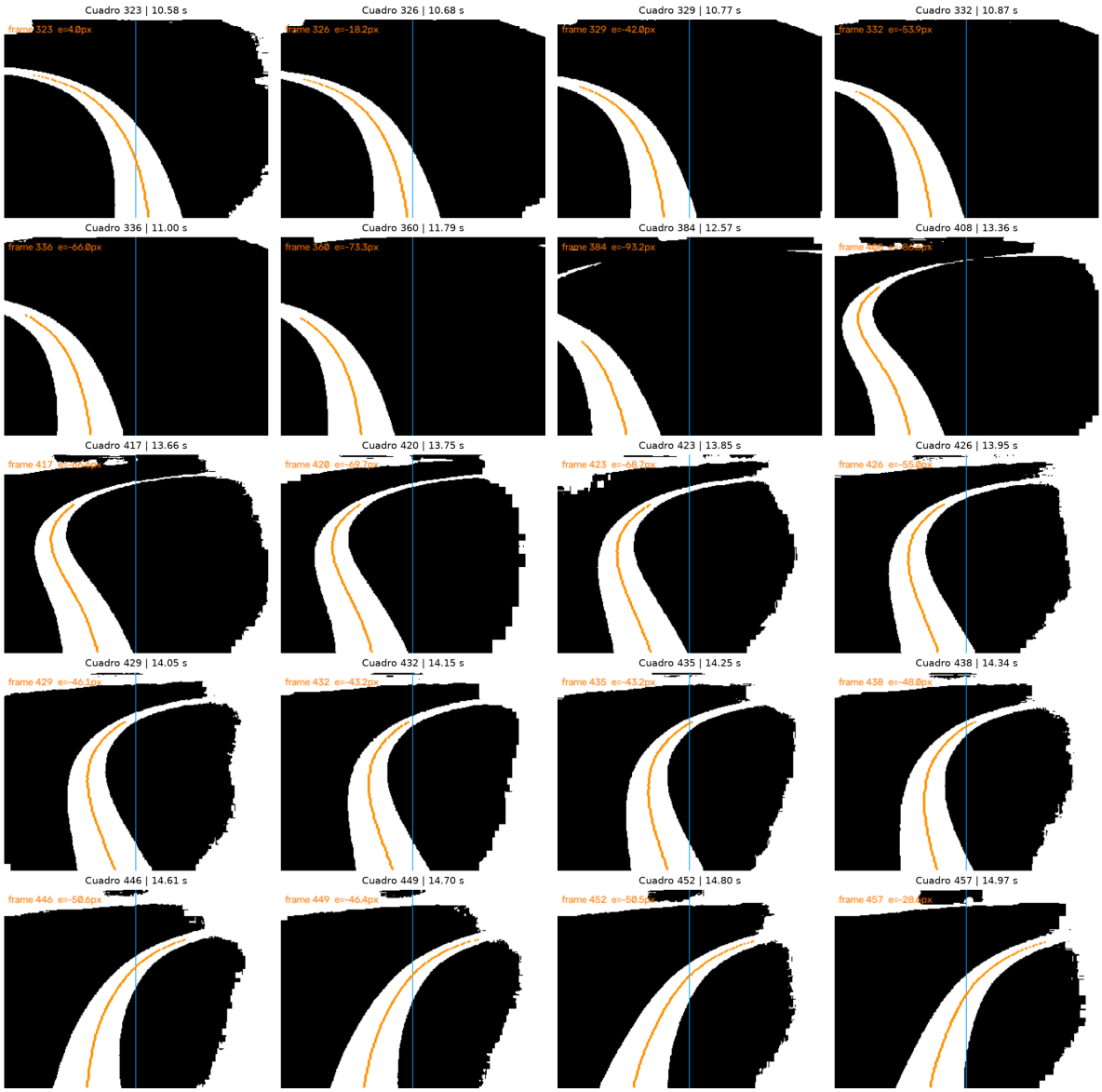
Se decodifica secuencialmente todo el clip. Se eligen 20 índices uniformes y 20 adicionales por curvatura aparente, con separación mínima de 3 índices respecto a los ya elegidos. Para cada trayectoria se ajusta x=a(y/H)^2+b(y/H)+c. El puntaje |a|·(rango(y/H))²/4 aproxima la desviación respecto a la cuerda; es un indicador en píxeles, no curvatura física. Se exige cobertura de al menos 0.3H filas para puntuar.

Los índices seleccionados son: 0, 24, 48, 56, 72, 96, 120, 124, 144, 168, 192, 216, 240, 264, 288, 306, 309, 312, 316, 320, 323, 326, 329, 332, 336, 360, 384, 408, 417, 420, 423, 426, 429, 432, 435, 438, 446, 449, 452, 457.

Se conservan PNG binarios sin pérdida y otros PNG con centros naranjas y referencia vertical azul. datos/seleccion_40.csv registra índice, tiempo y criterio; datos/exploracion_458.csv contiene el puntaje de todos los cuadros. Las siguientes dos hojas permiten revisar visualmente la selección.



2. Hoja de contacto: cuadros 21 a 40



La numeración de cada imagen corresponde al índice original del clip; las 40 imágenes no son una secuencia a cadencia uniforme. No se calcula velocidad diferenciando su posición en la lista.

6. Máscaras 3x3, ROI y distancias

Se usan K=unos(3,3) para apertura, cierre y gradiente morfológico, y Kx=[[-1,0,1],[-2,0,2],[-1,0,1]] para validar las transiciones horizontales. BORDER_REPLICATE extiende el valor del píxel del borde. La ROI empieza en 25% de la altura; la banda de medida/control comprende del 65% al 90%.

La referencia es x0=(W-1)/2=159.5 px. En cada fila: dL=x0-xL; dR=xR-x0; e=c-x0. Son distancias horizontales con signo y error lateral, no longitud recorrida. El ancho inclusivo difiere en un píxel de la separación geométrica entre extremos. centros_por_fila.csv conserva los valores para cada fila válida.

Se calcula además distancia euclidiana al borde más cercano con distanceTransform(255-borde, DIST_L2, DIST_MASK_PRECISE), evaluada en (floor((W-1)/2), round(0.8H)). Es una medida distinta del error horizontal: puede apuntar a un borde situado en otra fila. El borde tiene espesor discreto de la máscara 3x3; no hay precisión subpíxel.

En la banda de control hubo medida válida en 40/40 cuadros. El error lateral medio por cuadro varió entre -100.75 y 8.75 px. El signo negativo indica línea a la izquierda. La tabla completa está en datos/resultados_40.csv.

Conversión a centímetros pendiente: colocar una regla o patrón de tamaño conocido sobre el mismo plano de la pista, con la cámara en su montaje final. Para una sola fila, medir una longitud L_cm cuya proyección sea L_px y usar s(y)=L_cm/L_px; entonces e_cm=s(y)e_px. La escala cambia con la profundidad. Para un plano completo, usar al menos cuatro correspondencias no colineales y homografía a coordenadas métricas; corregir antes la distorsión mediante calibración intrínseca. No es válido asignar un ancho supuesto a la cinta.

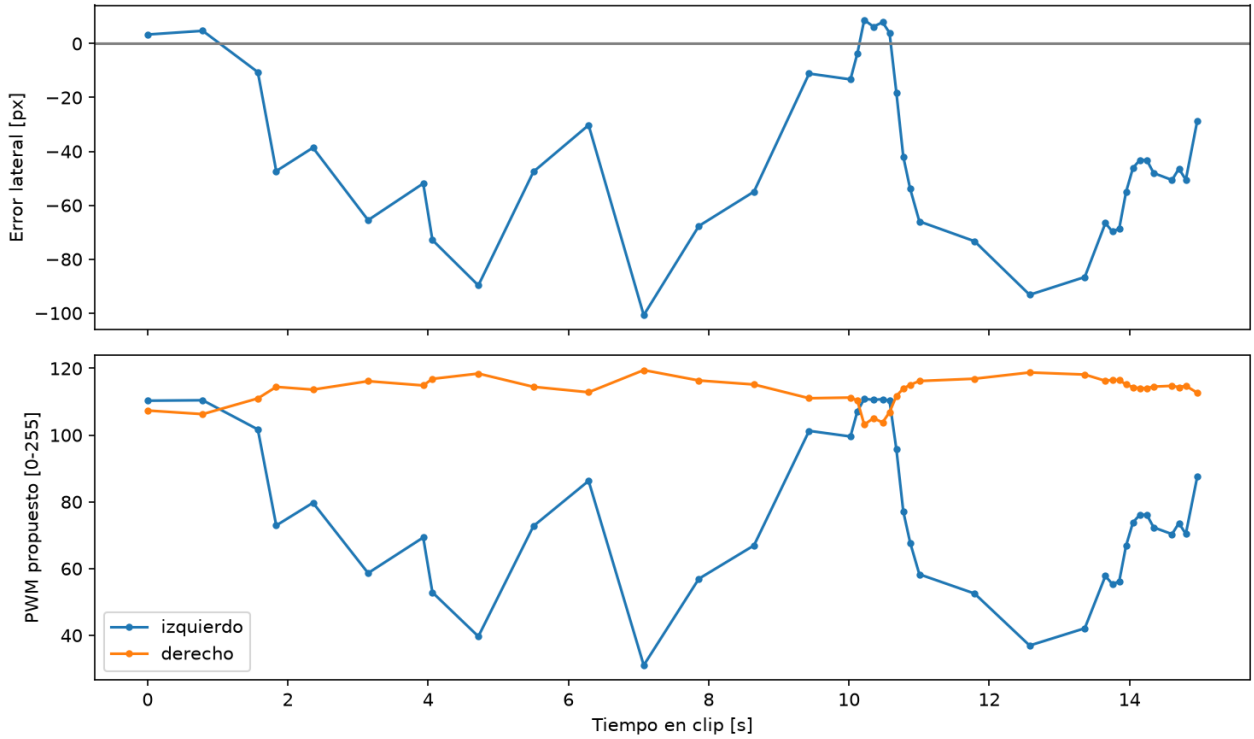
La distancia de la línea se interpreta aquí como separación lateral respecto al eje de cámara y distancia al borde, coherente con la finalidad de guiado. Para interpretar longitud de trayectoria o avance del móvil se necesita geometría y calibración adicionales.

7. Propuesta de PWM para los dos motores

Se promedia e(y) en la banda cercana y se normaliza en=e/159.5. Propuesta: base=110·(1-0.5·min(|en|,1)); u=70·en; PWM_L=clip(base+u,0,255), PWM_R=clip(base-u,0,255). Si la línea aparece a la derecha, se aumenta la rueda izquierda para girar a la derecha, suponiendo motores orientados para avance positivo. Verificar esa convención físicamente antes de usarla.

Los parámetros 110 y 70 son valores ilustrativos sin sintonización experimental. Si hay menos de 0.15H filas válidas en la banda, ambos PWM son cero. La reducción de base al crecer el error es una propuesta de moderación de velocidad; no prueba estabilidad ni mide velocidad. Añadir posteriormente límite de aceleración, compensación de zona muerta y control con encoders según el hardware.

7. Resultado de la propuesta, sin accionar motores



La gráfica muestra comandos calculados de los cuadros seleccionados. Las líneas conectan muestras no uniformes y no representan un ensayo continuo de control. No se comunica por puerto serial, red ni GPIO.

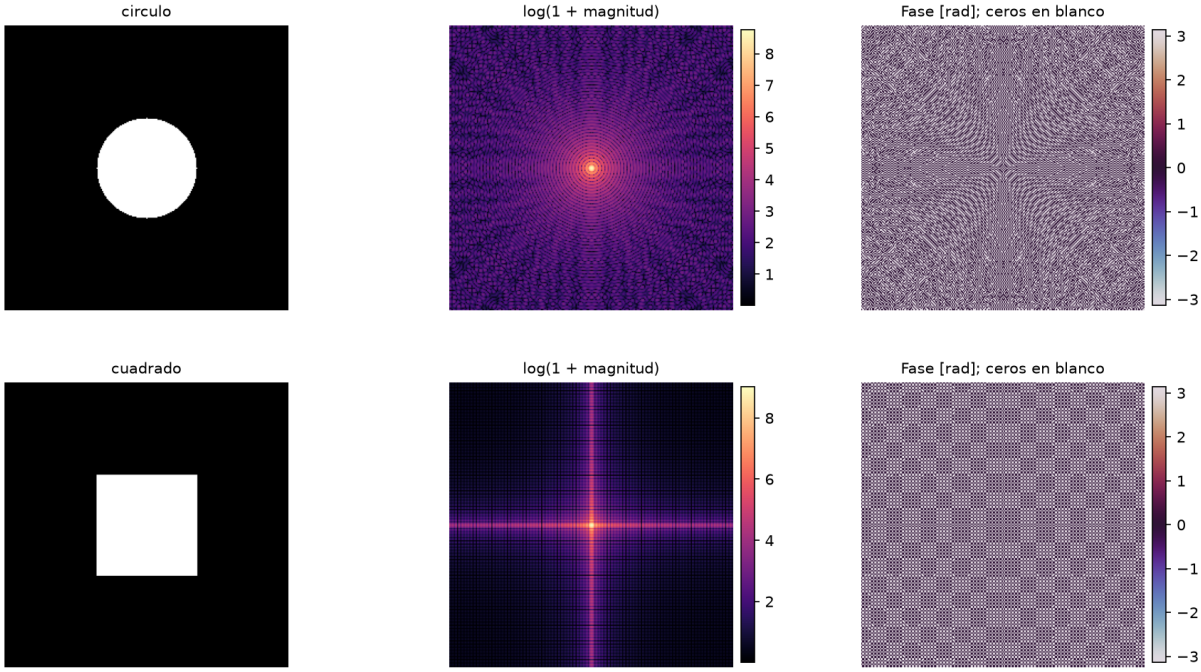
Calidad, supuestos y fallos posibles

La selección de la componente no resuelve automáticamente cruces, oclusiones ni sombras conectadas. Los umbrales de anchura y salto son heurísticos para 320×240. El recorte de ROI elimina el horizonte pero también ignora la pista lejana. Las filas descartadas no se interpolan y el archivo de resultados registra la cantidad de filas válidas.

La perspectiva transforma el movimiento de una cámara sobre el piso en algo más complejo que una traslación uniforme. Por eso Fourier permite estimar desplazamiento aparente entre imágenes, pero no directamente distancia recorrida por el móvil. Hace falta validación con marcas conocidas, homografía métrica y/o odometría.

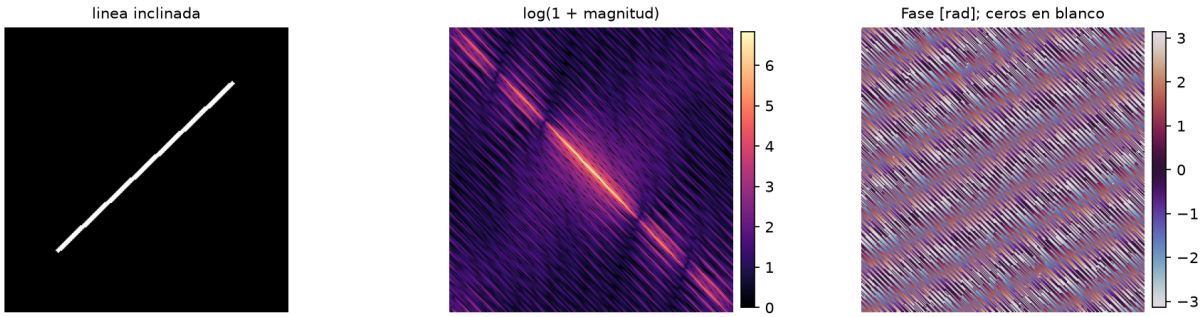
9. Fourier de figuras geométricas

Se generan imágenes monocromáticas de 256×256: círculo lleno de radio 45 px, cuadrado de 91×91 px y segmento inclinado de grosor 3 px. La intensidad es 0/1 para la DFT. Se guardan magnitud y fase numéricas en fourier/*.npz, sin desplazar sus índices. fftshift se aplica solo para visualizar el origen en el centro; la magnitud se muestra como log(1+|F|).

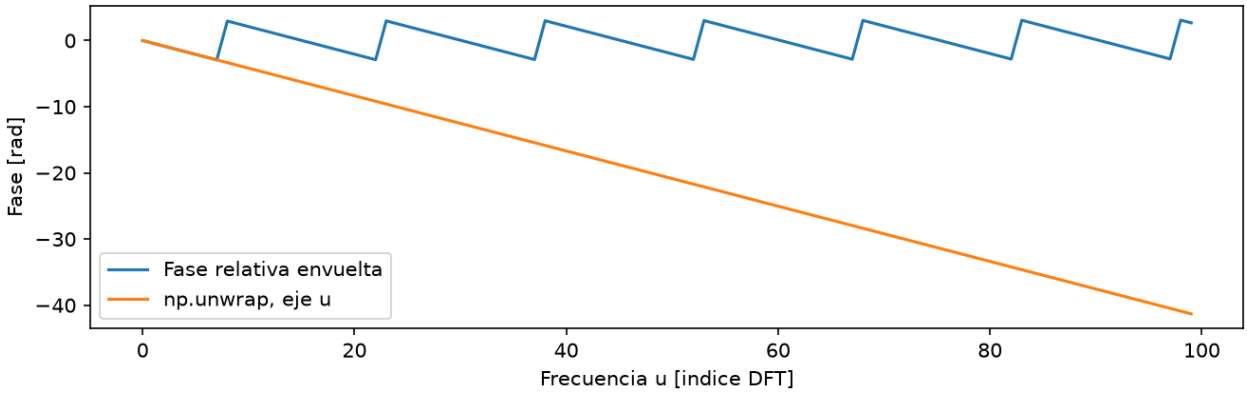


El círculo presenta simetría aproximadamente radial y lóbulos/anillos, alterados por el muestreo discreto. El cuadrado alineado con los ejes genera la estructura separable tipo sinc. La magnitud describe el contenido de frecuencias; no determina por sí sola la posición del objeto.

9. Línea inclinada, fase y unwrapping



El segmento finito produce concentración espectral en la dirección normal al segmento, con lóbulos por su longitud y grosor finitos. No es una delta ideal ni una línea infinitamente delgada. La fase conserva información de posición y estructura espacial, definida módulo 2π. La fase donde la magnitud es casi cero no es fiable: se ocultan esos valores en las figuras geométricas.



Ejemplo sintético explícito: se desplaza circularmente el cuadrado 17 px a la derecha. La diferencia máxima entre magnitudes es 9.09e-13; la fase relativa sigue una rampa -2πu·17/256 módulo 2π. np.unwrap sobre el eje de frecuencia recupera la continuidad de esa rampa. Este desplazamiento conocido no se presenta como resultado del video.

Unwrapping añade múltiplos de 2π para quitar saltos convencionales. No crea información donde la magnitud es nula, no elimina ruido y aplicar unwrap sucesivamente por filas y columnas no garantiza una solución bidimensional única. Para reconstruir con exp(j·fase) no se necesita desenvolver la fase.

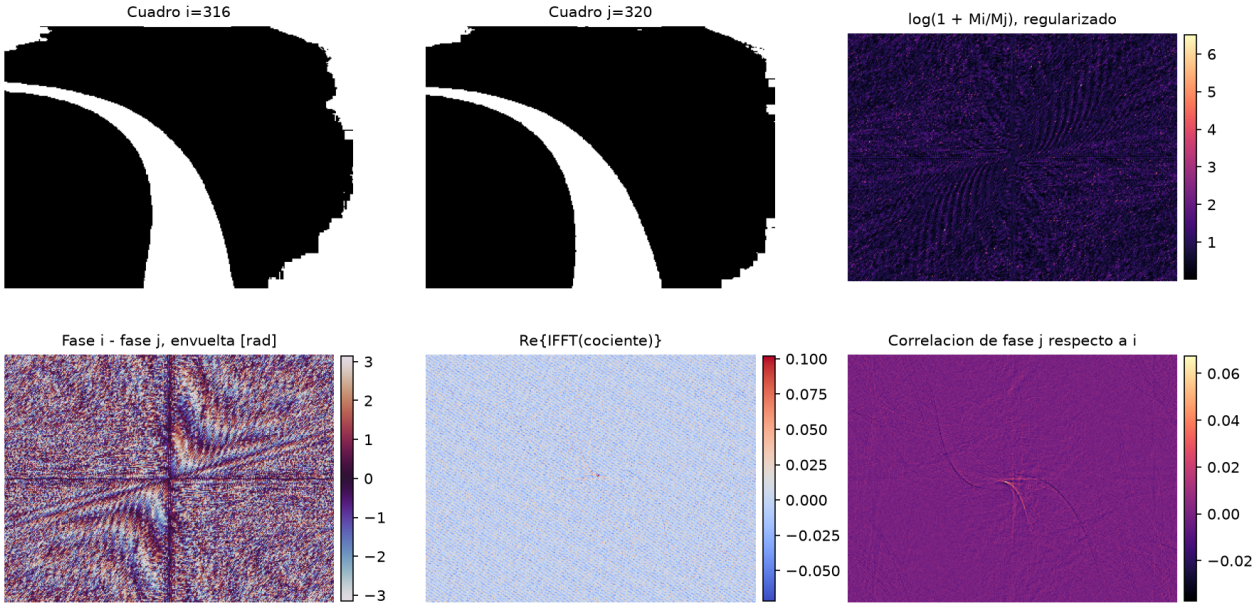
10. Fourier de 40 cuadros y operaciones entre un par

Para cada PNG binario se calcula F=fft2(B/255), M=abs(F) y fase=angle(F). Hay 40 archivos de matrices y dos atlas con magnitud/fase de todos los cuadros. No se usa la imagen logarítmica para operar: los cálculos usan matrices sin normalización visual.

Se usa i=316 y j=320, separados 4 índices (0.1310 s). Cumplen al menos tres cuadros intermedios. El par se elige entre los 40 extraídos, priorizando cercanía temporal y después el puntaje de curva de i.

a) R=Mi/Mj solo donde Mj>epsilon, con epsilon=10^-6·max(Mj)=0.01321; en los otros bins R=0. La fracción descartada es 0.0000%. Esta regularización evita divisiones por cero y amplificación arbitraria. La máscara de soporte también se guarda.

b) Se guarda fase_i-fase_j directa y también su equivalente envuelta angle(Fi·conj(Fj)). c) Se forma H=R·exp(j·delta_fase) y h=ifft2(H). H coincide con Fi/Fj en los bins admitidos: h es un núcleo de transferencia/deconvolución circular; no es automáticamente una imagen de la pista ni un mapa de distancias.



Comprobación numérica: reconstruir i mediante ifft2(H·Fj) da RMSE=1.27527e-16 en intensidad [0,1]. El residuo imaginario máximo de h es 3.4e-15. No se interpreta la parte imaginaria como una distancia.

10.d. Método para estimar desplazamiento

Se utiliza el teorema del desplazamiento: para j trasladado respecto a i, la fase cruzada contiene una rampa. Se calcula C=(Fj·conj(Fi))/|Fj·conj(Fi)| en frecuencias con energía y r=ifft2(C). El máximo de r da el desplazamiento circular de i hacia j, con coordenadas mayores a media imagen convertidas a signo negativo.

Pico sin ventana: dx=-7 px, dy=3 px. OpenCV phaseCorrelate con ventana Hann y estimación subpíxel: dx=-1.649 px, dy=4.575 px; respuesta=0.4472. Son desplazamientos aparentes de imagen y no medidas del recorrido físico. Las diferencias entre estimaciones evidencian sensibilidad a ventana, bordes y deformación de la escena.

La respuesta de phaseCorrelate es una medida de concentración del pico, no una probabilidad de acierto. Una pista repetitiva, curvada o con perspectiva puede producir máximos ambiguos. Para estimar distancia física: rectificar al plano del piso con homografía métrica, estimar traslación en regiones coherentes, rechazar pares con poca respuesta o residuos grandes y contrastar con una referencia independiente. También debe tenerse en cuenta que movimiento de escena y de cámara tienen sentidos opuestos.

Reproducción y archivos de evidencia

Desde taller3: python -m pip install -r requirements.txt ; luego python validar.py ; luego python taller.py. Cada comando se ejecuta por separado. Para otra salida: python taller.py --out outputs_nuevos. --video acepta un video ya umbralizado Otsu-inverso con blanco como línea; no acepta indistintamente video RGB crudo.

outputs/datos contiene selección, centros por fila, resultados, exploración y metadatos. outputs/cuadros contiene 40 binarios y 40 imágenes anotadas. outputs/fourier contiene magnitudes/fases de los 40 cuadros, las tres figuras y operaciones_par.npz. outputs/figuras contiene hojas de contacto, máscaras, control y atlas espectrales. informe.md es la versión editable generada de este texto.

Fuentes

Consigna: Taller_opencv3 (1).pdf, Control 2, dos páginas. Investigación previa: opencv_taller.pdf, Maikol Julián Pulido, 1 de octubre de 2026, cuatro páginas; se conserva a continuación.

OpenCV, Image Filtering (máscaras y bordes):
https://docs.opencv.org/2.4.13.7/modules/imgproc/doc/filtering.html

OpenCV, Miscellaneous Image Transformations (distancia al cero más próximo):
https://docs.opencv.org/doc/doxygen/html/d7/d1b/group__imgproc__misc.html

OpenCV, Motion Analysis and Object Tracking (phaseCorrelate):
https://docs.opencv.org/4.0.1/d7/df3/group__imgproc__motion.html

La documentación de versiones anteriores se utiliza para las definiciones matemáticas; las llamadas se ejecutaron y comprobaron en la versión de OpenCV indicada en este informe.

Taller de Visión Artificial – Control 2

Puntos de investigación

Maikol Julián Pulido
Ingeniería Electrónica – Escuela Colombiana de Ingeniería Julio Garavito
1 de octubre de 2026

3. ¿Cómo duplicar los píxeles del borde?

Para duplicar los píxeles del borde de una imagen se puede realizar una **extensión de la imagen**, añadiendo filas y columnas alrededor de sus límites. Los nuevos píxeles se generan tomando como referencia los valores de los píxeles que se encuentran más cerca del borde.

Una de las formas más sencillas consiste en utilizar una técnica de **replicación del borde**, en la cual el valor del píxel más cercano se repite hacia las nuevas posiciones.

Este procedimiento es útil cuando se desea aplicar un filtro o una máscara sobre los píxeles ubicados en los extremos de una imagen, ya que permite mantener el tamaño de la ventana de procesamiento incluso en los bordes.

4. ¿Cómo calcular la "distancia" entre píxeles en una imagen binarizada?

En una imagen binarizada se puede calcular la distancia entre dos píxeles utilizando una **transformada de distancia**. Las métricas más utilizadas son la distancia euclidiana, Manhattan y Chebyshev.

Si se tienen dos puntos:

$$P_1 = (x_1, y_1), \qquad P_2 = (x_2, y_2)$$

las principales distancias son:

- **Distancia euclidiana:**

  $$d_E = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

  Representa la distancia en línea recta entre los dos píxeles.

- **Distancia Manhattan:**

  $$d_M = |x_1 - x_2| + |y_1 - y_2|$$

  Calcula la distancia considerando únicamente desplazamientos horizontales y verticales.

- **Distancia de Chebyshev:**

$$d_C = \max\left(|x_1 - x_2|, |y_1 - y_2|\right)$$

  Corresponde al mayor desplazamiento entre las dos coordenadas.

También es posible calcular distancias directamente a partir de los índices de filas y columnas de la matriz que representa la imagen.

5. ¿Cómo definir una recta con puntos en la matriz de una imagen?

Para definir una recta en una imagen se seleccionan diferentes píxeles mediante sus coordenadas:

$$(x, y)$$

Estos puntos representan una aproximación discreta de una recta geométrica. Existen diferentes métodos para definir o encontrar una recta a partir de estos puntos.

a) Ecuación explícita y paramétrica

La recta puede definirse mediante su pendiente y un punto conocido:

$$y = mx + b$$

También puede utilizarse una representación paramétrica:

$$x = x_0 + at$$

$$y = y_0 + bt$$

Estas representaciones permiten determinar la trayectoria de una línea a partir de sus puntos.

b) Ajuste de líneas por mínimos cuadrados

Cuando se dispone de varios píxeles dispersos que pertenecen aproximadamente a un mismo borde, se puede utilizar el método de **mínimos cuadrados**. Este método busca la recta que mejor se ajusta al conjunto de puntos, minimizando el error entre los puntos observados y la recta estimada.

c) Transformada de Hough

La **transformada de Hough** permite detectar líneas a partir de los píxeles de una imagen.

En lugar de representar directamente la recta mediante las coordenadas cartesianas, se utiliza la representación:

$$\rho = x\cos\theta + y\sin\theta$$

donde ρ corresponde a la distancia perpendicular desde el origen hasta la recta y θ representa el ángulo de dicha perpendicular.

De esta manera, los puntos de la imagen se transforman en el espacio de parámetros (ρ, θ), permitiendo identificar líneas que comparten los mismos parámetros.

8. ¿En qué consiste la transformada de Fourier espacial en imágenes?

La **transformada de Fourier espacial** permite transformar una imagen desde el **dominio espacial** al **dominio de la frecuencia espacial**.

La idea principal es que una imagen puede representarse como una combinación de ondas sinusoidales bidimensionales con diferentes:

- amplitudes,
- frecuencias,
- fases.

En el dominio espacial se analiza directamente el valor de intensidad de cada píxel. En cambio, en el dominio de la frecuencia se analiza cómo se distribuyen las diferentes frecuencias espaciales presentes en la imagen.

Esto permite estudiar características como bordes, texturas y cambios de intensidad, además de facilitar diferentes operaciones de filtrado.

9. Transformada de un círculo, un cuadrado y una línea inclinada

Las características geométricas de un objeto en el dominio espacial producen patrones particulares en el dominio de Fourier.

- **Círculo:** su transformada presenta una magnitud con simetría circular, caracterizada por un lóbulo central y anillos concéntricos.
- **Cuadrado:** su transformada presenta una distribución asociada a la forma de la figura, con características alineadas con los ejes. Matemáticamente, puede relacionarse con el producto de dos funciones sinc.
- **Línea inclinada:** la transformada presenta una distribución concentrada sobre una línea cuya orientación es perpendicular a la orientación de la línea original.

¿Qué información proporcionan la magnitud y la fase?

La **magnitud** de la transformada de Fourier indica qué frecuencias espaciales están presentes en la imagen y con qué intensidad. Está relacionada con características como la forma, el tamaño y la textura.

La **fase** contiene información relacionada con la posición y estructura espacial de los objetos. Por ejemplo, un desplazamiento de un objeto puede producir principalmente un cambio en la fase de la transformada, mientras que la magnitud puede permanecer esencialmente igual.

¿Qué significa "unwrapping"?

La fase obtenida mediante la transformada de Fourier normalmente se representa en el intervalo:

$$(-\pi, \pi]$$

Por esta razón, pueden aparecer saltos de 2π entre valores consecutivos de fase.

El proceso denominado **phase unwrapping** o **desenvolvimiento de fase** consiste en agregar o restar múltiplos de 2π para eliminar estos saltos y obtener una representación de fase continua.